

Thiết kế bài toán và định nghĩa dữ liệu (Problem Framing and Data Definition)

PHỤ THUỘC B/03-giai-phau-he-thong (bộ ba đầu vào / đầu ra / tín hiệu giám sát); B/04-nen-tang-hoc-tu-du-lieu (identifiability và ba đòn bẩy — chương này biến đòn bẩy “thêm dữ liệu” thành một quyết định có con số).

TÓM TẮT

Chương này nói về những quyết định bạn phải ra *trước khi* có mô hình nào chạy: một mẫu là gì, nhân được định nghĩa thế nào ở các ca biên, gộp hay tách lớp, dùng *bbox* hay *mask*, lấy mẫu từ đâu, và nên đầu tư tiếp vào dữ liệu hay vào mô hình. Đọc xong bạn tự chẩn đoán được nguyên nhân của phần lớn thất bại triển khai, kể cả những nguyên nhân không hiện lên trên bất kỳ metric nội bộ nào. Bạn cũng trả lời được câu “gán nhãn thêm bao nhiêu thì đáng” bằng một phép tính hai dòng trên *learning curve*, thay vì bằng cảm tính. Nếu chỉ nhớ một điều: **đơn vị phân tích** (*unit of analysis*) và dạng đầu ra khoá chặt mọi thứ phía sau, còn thiên lệch sinh ra ở khâu thu thập thì không hàm mất mát nào sửa được — chỉ có thu thập lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tập dữ liệu** — nhãn là câu trả lời đúng gắn với một ảnh — tên lớp, tọa độ hộp, hay mask. Tập dữ liệu là tập các cặp (ảnh, nhãn) dùng để dạy và để đánh giá mô hình. Cả chương này nói về việc *định nghĩa* nhãn chứ không về việc học từ nó.
- **mẫu (sample) — hai nghĩa** — (a) một điểm dữ liệu đơn lẻ, “một mẫu ảnh”; (b) trong thống kê, cả tập được rút ra từ một quần thể, “mẫu gồm 500 ảnh”. Chương này dùng nghĩa (a) khi nói “số mẫu” và nghĩa (b) khi nói “lấy mẫu”; ngữ cảnh phân biệt được.
- **tập huấn luyện và tập kiểm thử** — tập dữ liệu được chia làm ít nhất hai phần: phần dùng để điều chỉnh tham số (huấn luyện) và phần giữ riêng, không cho mô hình nhìn thấy, dùng để ước lượng chất lượng trên dữ liệu mới (kiểm thử). Nếu một mẫu lọt vào cả hai phần thì con số kiểm thử lạc quan giả — hiện tượng gọi là rò rỉ dữ liệu.
- **các dạng nhãn không gian** — *bounding box* (bbox) là hình chữ nhật bao quanh vật, ghi bằng bốn số; *mask* là nhãn cho từng pixel, nói pixel đó thuộc vật nào; *polygon* là đường bao đa giác, xấp xỉ rẻ hơn của mask; *keypoint* là một tập điểm có ý nghĩa cố định (khớp vai, khuỷu tay); *hộp 3D* là hộp trong không gian ba chiều, cần thêm thông tin hình học.
- **lớp (class) và taxonomy** — lớp là một phạm trù mà nhãn có thể nhận (“xe”, “người”). Taxonomy là toàn bộ danh sách lớp cùng cách chúng gộp hay tách khỏi nhau — “xe” hay là “ô tô / xe tải / xe buýt”. Đây là một quyết định thiết kế, không phải một sự thật có sẵn trong thế giới.
- **quần thể và khung lấy mẫu** — quần thể (*population*) là toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Khung lấy mẫu (*sampling frame*) là phần của quần thể đó mà bạn thực sự chụp được. Hai thứ này gần như không bao giờ trùng nhau, và chênh lệch giữa chúng là chủ đề trung tâm của chương.
- **learning curve — hai nghĩa** — (a) đường cong sai số theo số vòng huấn luyện (epoch), dùng để xem mô hình đã hội tụ chưa; (b) đường cong sai số theo kích thước tập dữ liệu, dùng để quyết định có nên thu thập thêm dữ liệu không. Chương này chỉ dùng nghĩa (b), và đó là nghĩa ít người vẽ hơn hẳn.
- **IoU và mAP** — IoU (*intersection over union*) đo mức chồng lấn giữa dự đoán và nhãn: diện tích phần giao chia cho diện tích phần hợp, bằng 1 khi trùng khít. mAP là điểm tổng hợp cho bài

toán phát hiện, dựng trên IoU. Ở chương này chỉ cần biết chúng là các thước đo phụ thuộc vào dạng đầu ra bạn đã chọn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao hai người gán nhãn giỏi, cùng đọc một guideline, vẫn cho ra hai tập nhãn khác nhau — và vì sao con số bất đồng đó lại là trần hiệu năng của mô hình?
- Q2. Một mô hình ổn định qua năm lần chia tập rồi sập ngay tuần đầu triển khai. Nếu không phải overfitting thì là gì?
- Q3. Đổi từ bbox sang mask nghe như một thay đổi kiến trúc; vì sao nó thực ra là quyết định đắt nhất của cả dự án?
- Q4. Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” có trả lời được bằng một con số thay vì bằng thâm niên không?

1.1 Đặt vấn đề

Trước khi có mô hình, bạn phải biến một nhu cầu mơ hồ thành một bài toán học máy. Mọi sai lầm ở bước này không sửa được ở bước sau — đó là điều làm cho chương này khác hẳn mọi chương kỹ thuật khác trong cây. Một **hàm mất mát** (*loss function*) chọn sai thì đổi loss; một kiến trúc chọn sai thì huấn luyện lại; nhưng một đơn vị phân tích chọn sai, hay một dạng đầu ra chọn sai, thì phải gán nhãn lại từ đầu, và trong hầu hết dự án điều đó đồng nghĩa với việc dự án dừng.

Người mới thường coi phần này là “việc chuẩn bị” và muốn đi nhanh qua để tới phần thú vị. Quan sát lặp lại — và cũng là quan sát chung trong kỹ thuật hệ thống học máy [1] — là điều ngược lại: khi một dự án thị giác thất bại, nguyên nhân hiếm khi nằm ở kiến trúc. Nó thường nằm ở một trong hai chỗ: nhãn được định nghĩa không nhất quán, hoặc dữ liệu được thu thập từ một quần thể khác với quần thể sẽ dùng hệ thống. Cả hai nguyên nhân này đều *vô hình* đối với mọi metric nội bộ: bạn có thể đạt **mAP** rất cao trên tập kiểm thử của chính mình và vẫn thất bại hoàn toàn khi triển khai, mà không con số nào trong bảng cảnh báo bạn.

Nếu bạn đã làm **benchmark** SLAM thì bạn có sẵn trực giác đúng cho chương này, chỉ là dưới tên khác. Một quỹ đạo **chân trị** (*ground truth*) trên EuRoC được định nghĩa trong một hệ quy chiếu cụ thể; nếu bạn đánh giá quỹ đạo camera bằng chân trị của thân máy bay, mọi con số đều ra, đều đẹp, và đều sai. Sai lầm không nằm ở thuật toán mà nằm ở định nghĩa của đại lượng được đo. Chương này là phiên bản tổng quát của bài học đó, áp cho toàn bộ khâu thiết kế bài toán.

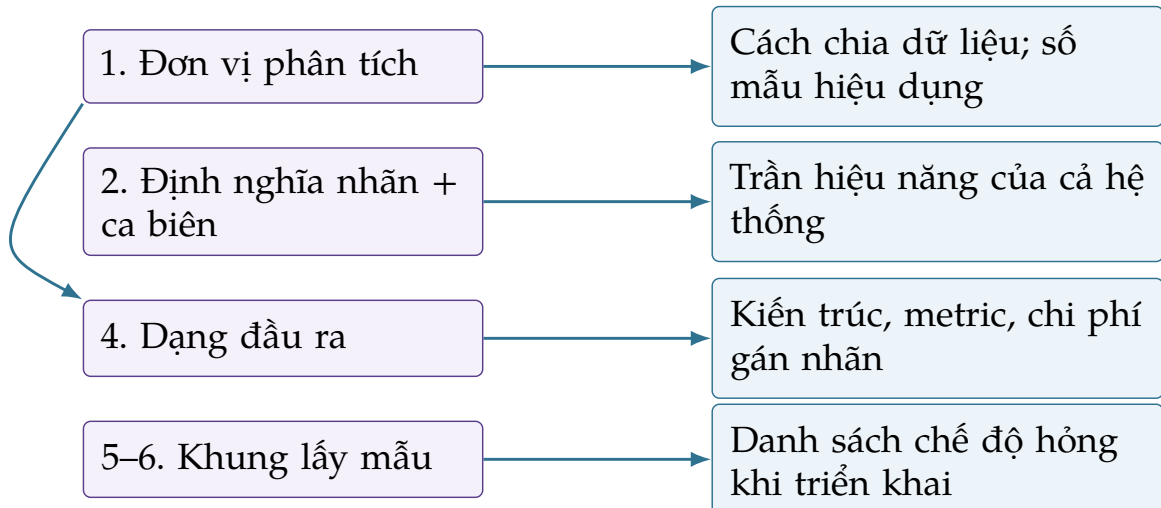
TRỰC GIÁC

Định nghĩa bài toán giống việc đúc một cái khuôn: mọi thứ đổ vào sau đó đều mang hình dạng của nó. Chọn đơn vị phân tích là chọn kích thước khuôn; chọn dạng đầu ra là chọn hình dạng khoang; định nghĩa nhãn kèm ca biên là mài các cạnh cho hai người thợ khác nhau đúc ra cùng một chi tiết. Bạn có thể đổi kim loại đổ vào — đổi kiến trúc, đổi loss, đổi **bộ tối ưu** (*optimizer*) — bao nhiêu lần tùy thích, nhưng đổi khuôn thì phải phá cả mẻ đã đúc.

1.2 Tám quyết định theo thứ tự ràng buộc (Eight Decisions, Ordered by Reach)

A. Quyết định nào khoá cái gì

Thứ tự của mạch dưới đây đi từ quyết định ràng buộc nhiều thứ nhất tới quyết định ràng buộc ít nhất. Đảo ngược thứ tự là sai lầm phổ biến: chọn dạng đầu ra trước khi biết đơn vị phân tích là gì thì bạn sẽ phát hiện mâu thuẫn sau khi đã gán vài nghìn ảnh. Hình 1.1 cho thấy tầm với của từng quyết định — đọc nó trước sẽ tiết kiệm cho bạn cả mạch phía sau.



Hình 1.1. Bốn quyết định có tầm với xa nhất và thứ mà mỗi cái khoá lại. Mũi tên cong bên trái nhắc rằng đơn vị phân tích còn ràng buộc cả dạng đầu ra: chọn video làm đơn vị thì nhãn theo khung hình mất phần lớn ý nghĩa.

B. Tám bước

I Mạch thiết kế một bài toán thị giác

1 Vấn đề	“một mẫu” là gì, câu hỏi tưởng như không cần hỏi.
Giải pháp	chọn đơn vị phân tích một cách có ý thức — một khung hình, một video, một bệnh nhân, một ca chụp, một chuyến đi.
Hệ quả	nó quyết định luôn cách chia dữ liệu ở chương 7: chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ dữ liệu (<i>data leakage</i>). Hai khung hình cách nhau 33 ms gần như là cùng một ảnh, nên chúng sẽ nằm ở cả tập huấn luyện lẫn tập kiểm thử.
Giá phải trả	chọn đơn vị lớn hơn làm số mẫu hiệu dụng giảm mạnh, và bạn phải chấp nhận con số đó thay vì con số đẹp hơn nhưng vô nghĩa. ↪ tiền đề B/07 — quy tắc chia dữ liệu ở chương đánh giá chỉ đúng khi đơn vị phân tích đã được chọn đúng ở đây
2 Vấn đề	nhãn nghe có vẻ hiển nhiên nhưng không phải.
Giả định	phần lớn cái ta gọi là “mô hình kém” thực ra là nhãn được định nghĩa không nhất quán: biên của mask đặt ở đâu khi vật nhoè, che khuất bao nhiêu phần trăm thì vẫn gán, vật nhỏ tới mức nào thì bỏ qua.
Giải pháp	viết định nghĩa nhãn kèm các ca biên <i>trước khi</i> gán mẫu đầu tiên, rồi thử nghiệm nó trên vài chục ảnh với hai người gán nhãn độc lập.
Hệ quả	mức đồng thuận giữa người gán nhãn (<i>inter-annotator agreement</i>) là trần hiệu năng thực tế của hệ thống; không mô hình nào vượt qua độ nhất quán của dữ liệu dạy nó một cách đáng tin.

3 Vấn đề

Giải pháp

Giá phải trả

Hệ quả

chia lớp thể nào cho đúng.

thiết kế *taxonomy*, tức quyết định gộp hay tách.

gộp làm bài toán dễ và metric đẹp nhưng đầu ra kém hữu ích; tách làm đầu ra hữu ích hơn nhưng đẩy chi phí gán nhãn lên và làm các lớp hiếm quá thừa để học.

quy tắc thực dụng: tách khi và chỉ khi hai lớp dẫn tới hai *hành động* khác nhau ở phía sau hệ thống; nếu hệ thống phản ứng như nhau thì việc tách chỉ tạo công gán nhãn và bất đồng giữa người gán nhãn.

4 Vấn đề

Giải pháp

Giá phải trả

Hệ quả

chọn dạng đầu ra nào.

cân nhắc giữa nhãn ảnh, bbox, mask, *polygon*, *keypoint*, **hộp 3D** (*3D box*) — xem Bảng 1.1.

đây là quyết định đắt nhất trong cả dự án, vì nó ràng buộc *đồng thời* kiến trúc, metric và chi phí gán nhãn. Đổi từ bbox sang mask giữa chừng nghĩa là gán lại từ đầu.

nên trả giá suy nghĩ ở đây bằng một thử nghiệm nhỏ, đừng trả giá bằng nửa ngân sách gán nhãn.

5 Vấn đề

Giải pháp

Giả định

Hệ quả

bạn huấn luyện trên tập nào và triển khai cho ai?

phân biệt rành mạch **quần thể** (*population*) — toàn bộ tình huống hệ thống sẽ gặp khi chạy thật — với **khung lấy mẫu** (*sampling frame*), phần của quần thể đó mà bạn thực sự lấy mẫu được.

hai thứ trùng nhau, một giả định gần như luôn sai.

khoảng cách giữa chúng là nguồn thất bại số một khi triển khai, và nó không hiện lên trên bất kỳ metric nội bộ nào vì tập kiểm thử của bạn cũng rút từ chính khung lấy mẫu.

6 Vấn đề

Giả định

Giải pháp

Hệ quả

thiên lệch được cài vào ngay từ khâu đầu.

selection bias khi thu thập: bạn chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba, lấy ảnh ở những địa điểm cho phép quay.

nguyên tắc cứng — thiên lệch sinh ra ở khâu thu thập thì không mô hình, loss, hay *reweighting* nào sửa được ở khâu cuối. Reweighting chỉ sửa được lệch *tỉ lệ* giữa các nhóm đã có mặt; nó không tạo ra được nhóm chưa từng xuất hiện.

chỉ có thu thập lại, nên chi phí của một quyết định lấy mẫu草 thả là toàn bộ ngân sách dữ liệu.

7 Vấn đề

Giải pháp

Hệ quả

nên đầu tư vào dữ liệu hay vào mô hình?

một phép đo thay cho tranh luận — vẽ learning curve theo kích thước dữ liệu: huấn luyện cùng một cấu hình ở 10%, 25%, 50%, 100% dữ liệu và nhìn hình dạng đường cong.

curve còn dốc thì thu thập tiếp; curve đã phẳng thì tiền gán nhãn tiếp theo gần như bị đốt, và bạn phải đổi mô hình, đổi dạng đầu ra, hoặc đổi bài toán. Đây chính là đòn bẩy “thêm dữ liệu” của chương 4, lần này có quy trình đo cụ thể.

↪ **hệ quả của** B/04 — bảng ba đòn bẩy nói có ba lựa chọn; learning curve nói *chọn cái nào*

8 Vấn đề

Giải pháp

Hệ quả

dữ liệu không miễn phí, mà thảo luận về nó thường không có đơn vị.

so *chi phí thu thập* với *giá trị biên mỗi mẫu* đọc từ chính đường cong ở bước trên.

nó biến một quyết định kỹ thuật thành quyết định kinh tế phát biểu được bằng con số — cách nghĩ dùng được ở mọi dự án thật, kể cả khi con số chỉ chính xác tới một bậc độ lớn.

I 1.3 Chọn dạng đầu ra (Choosing the Output Representation)

Trong tám bước trên, bước bốn đáng được mở rộng vì nó là bước duy nhất mà sai lầm không có đường lùi rẽ. Cột chi phí trong Bảng 1.1 là *tương đối* và chỉ nên đọc ở mức bậc độ lớn: các con số này phản ánh kinh nghiệm thực hành lưu truyền trong cộng đồng gắn nhãn, không phải một phép đo có kiểm soát. Chúng còn thay đổi mạnh khi có trợ lý gắn nhãn bằng mô hình nền tảng.

Bảng 1.1. Chọn dạng đầu ra. Cột “hóng khi nào” là cột quyết định: nó cho biết loại lỗi bạn tự nguyện chấp nhận vĩnh viễn khi chọn dạng đó.

Dạng	Giả định về bài toán	Chi phí nhãn (tương đối)	Hóng khi nào
Nhãn ảnh	Chỉ cần biết trong ảnh có gì	1×	Khi hệ thống cần định vị hoặc đếm
Bbox	Vật tách rời, hình chữ nhật đủ mô tả nó	5–10×	Vật dài, cong, hoặc chồng lấn: hộp chứa chủ yếu là nền
Mask	Cần biên chính xác tới mức pixel	20–50×	Biên mơ hồ (tóc, khối, kính) gây bất đồng lớn giữa người gắn nhãn
Polygon	Cần biên nhưng chấp nhận xấp xỉ đa giác	10–30×	Biên phức tạp bị làm trơn quá mức
Keypoint	Vật có cấu trúc cố định đã biết trước	10–20×	Vật không có cấu trúc chung, hoặc bị che một phần
Hộp 3D	Có hình học hoặc cảm biến phụ đồng bộ	50× trở lên	Lệch hiệu chuẩn (<i>calibration</i>) hoặc lệch thời gian giữa các cảm biến

Cách chọn đúng không phải là “chọn dạng giàu thông tin nhất mà ngân sách cho phép”, mà là đi ngược từ hành động phía sau: hệ thống của bạn *làm gì* với đầu ra? Nếu robot chỉ cần biết “có chướng ngại trong hành lang bên trái hay không” thì mask là lãng phí nhiều chục lần và còn làm chậm cả pipeline. Nếu nó phải bám theo mép một vật dài và cong thì bbox *sai về nguyên tắc*, không chỉ kém chính xác. Hình chữ nhật bao quanh một vật nằm chéo chứa phần lớn diện tích là nền. Hậu quả kép: bộ điều khiển phía sau không dùng được dự đoán đó, còn mọi metric dựa trên *IoU* thì vẫn khen nó.

↔ **tiền đề** B/03 — đây chính là câu hỏi “đầu ra là gì của quy trình bảy câu hỏi, lần này phải trả lời cho hệ thống của chính bạn

Một dấu hiệu nhận biết mình chọn sai dạng đầu ra: bạn phải viết ngày càng nhiều luật hậu xử lý để dịch đầu ra của mô hình thành thứ hệ thống thật cần.

I 1.4 Quần thể và khung lấy mẫu (Population vs Sampling Frame)

Bước năm và sáu nói cùng một chuyện từ hai phía, và đáng được tách ra vì chúng là nguồn thất bại triển khai hay gặp nhất. Nói gọn thì có ba tập lồng nhau, không phải hai: quần thể là tập tất cả tình huống hệ thống sẽ gặp; khung lấy mẫu là tập tình huống bạn có khả năng chụp được; tập dữ liệu là một mẫu rút từ khung lấy mẫu, *không phải* từ quần thể. Tập kiểm thử của bạn cũng rút từ khung lấy mẫu, nên nó đo được sai số trong khung và mù hoàn toàn với phần quần thể nằm ngoài khung.

Điều này giải thích một hiện tượng gây bối rối: mô hình đạt kết quả tốt và ổn định qua nhiều lần chia tập, rồi sập khi triển khai. Không có gì mâu thuẫn — mọi phép chia đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ nhiều lần thay vì đo thêm thứ mới. Các

benchmark có kiểm soát về dịch chuyển phân phối được xây dựng chính vì lý do này [2], [3]. Bài học chung từ chúng: độ chính xác trong phân phối không dự đoán tốt độ chính xác ngoài phân phối.

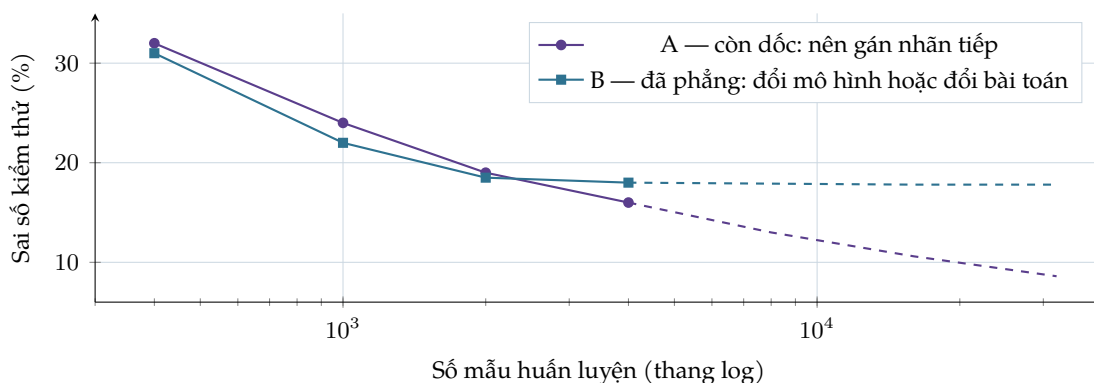
↔ **tiền đề** B/08 — chênh lệch quần thể ↔ khung lấy mẫu là dạng tĩnh của dịch chuyển phân phối, chỉ khác ở chỗ nó có mặt ngay từ ngày đầu

Hệ quả hành động thì đơn giản và không dễ chịu. Trước khi thu thập, hãy viết ra bằng lời quần thể mục tiêu và liệt kê các trục biến thiên của nó: thiết bị, ánh sáng, mùa, địa điểm, người dùng. Sau đó đánh dấu trục nào khung lấy mẫu của bạn không phủ. Danh sách đó chính là danh sách chế độ hỏng đã biết trước của hệ thống, và nó nên nằm trong *datasheet* của tập dữ liệu [4] chứ không nằm trong đầu một người. Nếu một trục quan trọng không phủ được, quyết định đúng là thu hẹp phạm vi cam kết của sản phẩm, chứ không phải hy vọng mô hình tự tổng quát hoá.

1.5 Đường cong học theo kích thước dữ liệu (Data-Size Learning Curve)

A. Hai hình dạng, hai quyết định trái ngược

Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” thường được tranh luận bằng trực giác và thâm niên. Nó không cần phải như vậy: một buổi huấn luyện ở bốn tỉ lệ dữ liệu cho bạn câu trả lời kiểm chứng được. Điều quan trọng là bạn phân biệt hai trường hợp bằng *hình dạng* đường cong chứ không bằng một con số đơn lẻ (Hình 1.2).



Hình 1.2. Hai hình dạng learning curve theo kích thước dữ liệu, vẽ trên trục log. Đường liền là bốn điểm đo được ở 10%, 25%, 50%, 100% dữ liệu; đường đứt là ngoại suy. Đường A gần thẳng trên trục log — dấu hiệu quy luật lũy thừa còn hiệu lực; đường B đã chạm trần, và trần đó thường là mức nhất quán của nhãn chứ không phải giới hạn của mô hình. Số liệu là minh họa, không phải phép đo trên một tập cụ thể.

B. Phép tính hai dòng biến nó thành quyết định kinh tế

Với đường A, sai số giảm từ 24% ở 1000 mẫu xuống 16% ở 4000 mẫu. Giả sử dạng lũy thừa $e(n) \approx a n^{-b}$, lấy tỉ số hai điểm ta có

$$b = \frac{\log(24/16)}{\log(4000/1000)} = \frac{\log 1,5}{\log 4} \approx \frac{0,405}{1,386} \approx 0,29.$$

Số mũ b nói một điều duy nhất: *mỗi lần nhân đôi dữ liệu thì sai số nhân với 2^{-b}* . Với $b \approx 0,29$, nhân tư số mẫu lên 16,000 cho $e \approx 16 \cdot 4^{-0,29} \approx 10,6\%$, nghĩa là 12,000 ảnh gán nhãn thêm mua được khoảng 5,4 điểm. Nếu chi phí gán một ảnh là c thì bạn đang cân nhắc bỏ 12,000 c để đổi lấy 5,4 điểm, và bạn so nó với các phương án khác — thuê thêm một kỹ sư trong một tháng để thử kiến trúc mới, hay giảm c bằng trợ lý gán nhãn tự động. Với đường B thì phép tính không cần làm: b gần 0, nên mọi khoản đầu tư dữ liệu đều trả về gần 0.

Hai cảnh báo về công cụ này. Thứ nhất, ngoại suy quy luật lũy thừa ra ngoài dải đã đo là suy đoán và thường lạc quan quá mức, vì đường cong thật cuối cùng cũng phải chạm trần nhãn; hãy coi 10,6% là cận trên của kỳ vọng chứ không phải một dự báo. Thứ hai, mỗi điểm phải được huấn luyện với cùng một quy trình và có phương sai được ước lượng, nếu không bạn đang đọc nhiều như đọc xu hướng — đây chính là bài học từ benchmark có thành phần ngẫu nhiên mà bạn đã gặp ở phía SLAM.

1.6 Thực hành

Chương này chỉ chuyển thành kỹ năng khi bạn tự chịu hậu quả của một định nghĩa nhãn lỏng lẻo. Vì vậy cả hai bài tập đều xoay quanh một phép đo: mức bất đồng giữa hai người gán nhãn. Đó là con số hầu như không dự án nào đo, mà hầu như dự án nào cũng bị nó chặn.

Repo và tài nguyên. Ba nhóm công cụ:

- **Gán nhãn** — [HumanSignal/label-studio](#), [cvat-ai/cvat](#), Roboflow Annotate.
- **Trợ lý gán nhãn bằng mô hình nền tảng** — dòng SAM cho mask theo *prompt* [5] và dòng Grounding DINO cho box theo mô tả văn bản. Chúng làm dịch chuyển đáng kể cột chi phí trong Bảng 1.1, nhưng *không* làm dịch chuyển vấn đề nhất quán của định nghĩa nhãn.
- **Soát và tài liệu hoá dữ liệu** — [voxel51/fiftyone](#) để khám phá; datasheets [4] và *model cards* [6] để viết tài liệu kèm.

Danh sách tên mô hình chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; CVAT và FiftyOne bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trong bài toán của bạn, hai người gán nhãn giỏi sẽ bất đồng ở đâu — và bạn xử lý bằng cách siết guideline hay bằng cách đổi định nghĩa nhãn? Nếu chuyển đơn vị phân tích từ khung hình sang video thì tập kiểm thử phải chia lại thế nào, và số mẫu hiệu dụng thay đổi bao nhiêu lần? Quần thể mục tiêu của bạn có trực biến thiên nào mà khung lấy mẫu hiện tại không phủ?

Bài tập phụ. Nếu bài toán của bạn không phải SLAM, phiên bản tổng quát của bài tập này là: viết *annotation guideline* hai trang, tự gán 200 ảnh, rồi nhờ một người khác gán 50 ảnh trong số đó. Đo mức bất đồng bằng **hệ số kappa của Cohen** (*Cohen's kappa*) [7] nếu là bài toán phân loại, hoặc bằng tỉ lệ khớp ở một ngưỡng IoU cố định nếu là phát hiện. Mini project SLAM ở cuối chương là phiên bản gắn với hệ thống của bạn.

1.7 Giới hạn và trường hợp hỏng

- **Khung này giả định bạn có quyền quyết định về dữ liệu.** Rất nhiều dự án bắt đầu từ một tập đã tồn tại với nhãn đã gán; khi đó phần lớn tám bước trở thành công việc khảo cổ — đọc ngược từ dữ liệu ra định nghĩa nhãn ngầm mà người trước đã dùng. Việc này vẫn đáng làm và thường phát hiện những mâu thuẫn giải thích được vì sao mô hình kém, nhưng nó không thay thế được quyền thiết kế.
- **Nó giả định quần thể mục tiêu đứng yên đủ lâu.** Có những hệ thống mà môi trường đổi theo mùa, theo phiên bản phần cứng, hay theo hành vi người dùng. Ở đó “quần thể” là một mục tiêu di động. Bài toán đúng không phải thiết kế một lần cho tốt, mà là dựng một vòng lặp thu thập lại; đó là nội dung của B/08-dich-chuyen-phan-phoi.
- **Mức đồng thuận là trần thực dụng, không phải trần lý thuyết.** Có trường hợp mô hình vượt từng người gán nhãn đơn lẻ vì nó học phần tín hiệu chung và trung bình hoá phần nhiễu riêng của mỗi người; ngược lại, có những tập dữ liệu nổi tiếng mà chính nhãn kiểm thử chứa

lượng lỗi đủ lớn để làm sai lệch bảng xếp hạng [8]. Hãy dùng nó như tín hiệu chẩn đoán mạnh, đừng dùng như hằng số vật lý.

CẠM BẤY

Cạm bẫy tốn kém nhất trong chương này là tin rằng một thiên lệch thu thập có thể sửa được ở khâu huấn luyện. Reweighting, *resampling* hay các hàm mất mát cân bằng nhóm chỉ điều chỉnh được *tỉ lệ* giữa những nhóm đã có mặt trong dữ liệu. Giả sử hệ thống sẽ gặp điều kiện ban đêm mà tập dữ liệu không có một ảnh ban đêm nào. Không trọng số nào tạo ra được thông tin đó. Mô hình sẽ đưa ra dự đoán tự tin và sai, còn tập kiểm thử của bạn thì không hề một lời. Dấu hiệu nhận biết: cuộc thảo luận về “xử lý mất cân bằng dữ liệu” diễn ra hoàn toàn bằng ngôn ngữ hàm mất mát, không ai nhắc tới việc đi chụp thêm.

CẦN NHỚ

Đơn vị phân tích và dạng đầu ra là hai quyết định khoá chặt mọi thứ phía sau: sai ở đó thì phải gán nhãn lại từ đầu. Thiên lệch sinh ra ở khâu thu thập chỉ sửa được bằng cách thu thập lại — không loss nào, không trọng số nào chạm tới nó. Và trước khi tranh luận “thêm dữ liệu hay đổi mô hình”, hãy vẽ learning curve: số mũ b trả lời câu hỏi đó bằng một con số. [T1]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Đơn vị phân tích	Thứ được tính là “một mẫu”: một khung hình, một video, một bệnh nhân, một chuyến đi. Nó quyết định luôn cách chia dữ liệu và số mẫu hiệu dụng, nên chọn sai là phải làm lại từ đầu.
••• Rò rỉ dữ liệu	Cùng một thông tin có mặt ở cả tập huấn luyện lẫn tập kiểm thử, làm con số kiểm thử lạc quan giả. Chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo ra nó.
••• Ca biên	Các trường hợp lấp lửng trong định nghĩa nhãn: biên vật nhoè, mức che khuất, kích thước tối thiểu. Phải viết ra giấy trước khi gán mẫu đầu tiên, vì sau đó sửa nghĩa là gán lại.
••• Mức đồng thuận giữa người gán nhãn	Tỉ lệ hai người gán độc lập cho ra cùng một nhãn. Đây là trần hiệu năng thực tế của hệ thống, vì mô hình chỉ học được từ những nhãn họ tạo ra.
•• Khi nào mô hình vượt được trần đó?	Khi mỗi người nhầm theo một kiểu riêng. Mô hình học phần tín hiệu chung và trung bình hoá phần nhiễu riêng của từng người, nên nhỉnh hơn từng người đơn lẻ.
•• Taxonomy	Danh sách lớp cùng cách chúng gộp hay tách. Quy tắc thực dụng: chỉ tách khi hai lớp dẫn tới hai hành động khác nhau ở phía sau hệ thống.
••• Dạng đầu ra	Nhãn ảnh, bbox, mask, polygon, keypoint, hay hộp 3D. Đây là quyết định đắt nhất dự án vì nó ràng buộc đồng thời kiến trúc, metric và chi phí gán nhãn.
••• Vì sao đổi dạng đầu ra giữa chừng lại đắt tới vậy?	Kiến trúc thì huấn luyện lại là xong, nhưng nhãn phải gán lại toàn bộ từ đầu. Đổi từ bbox sang mask là nhãn chỉ phí gán nhãn lên vài lần.
•• Bbox	Hình chữ nhật bao quanh vật, ghi bằng bốn số. Nó sai về nguyên tắc với vật dài và cong, vì hộp bao một vật nằm chéo chứa phần lớn diện tích là nền.
•• Bạn phải viết ngày càng nhiều luật hậu xử lý. Triệu chứng của gì?	Dạng đầu ra chọn sai: bạn đang dịch đầu ra của mô hình thành thứ hệ thống thật cần. Phép phân biệt là hỏi hệ thống làm gì với đầu ra, rồi chọn dạng đi ngược từ đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Quần thể	Toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Đây là thứ bạn cam kết phục vụ, không phải thứ bạn đo được.
●●● Khung lấy mẫu	Phần của quần thể mà bạn thực sự chụp được. Tập kiểm thử cũng rút từ đây, nên nó mù hoàn toàn với phần quần thể nằm ngoài khung.
●●● Ổn định qua năm lần chia tập rồi sập khi triển khai. Nguyên nhân?	Chênh lệch giữa quần thể và khung lấy mẫu, không phải overfitting. Cả năm phép chia đều nằm trong cùng một khung nên chúng đo lại cùng một thứ; phép phân biệt là dựng một tập kiểm thử từ ngoài khung.
●●● Selection bias	Thiên lệch cài vào ngay khâu thu thập: chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba. Nó không sửa được ở khâu cuối, chỉ có thu thập lại.
●●● Vì sao reweighting không sửa được selection bias?	Reweighting chỉ chỉnh tỉ lệ giữa những nhóm đã có mặt trong dữ liệu. Nó không tạo ra được nhóm chưa từng xuất hiện, ví dụ ảnh ban đêm khi tập dữ liệu không có tấm nào.
●● Cả nhóm bàn “mất cân bằng dữ liệu” hoàn toàn bằng ngôn ngữ hàm mất mát	Dấu hiệu đang rơi vào bẫy reweighting. Phép kiểm tra: hỏi trong dữ liệu có bao nhiêu mẫu của nhóm thiểu; nếu là không thì việc phải làm là đi chụp thêm.
●●● Learning curve theo kích thước dữ liệu	Sai số kiểm thử vẽ theo số mẫu huấn luyện, đo ở 10, 25, 50 và 100 phần trăm dữ liệu. Nó biến câu “dữ liệu hay mô hình” thành một con số thay vì một cuộc tranh luận theo thâm niên.
●● Số mũ b của learning curve	Độ dốc trên trục log: mỗi lần nhân đôi dữ liệu thì sai số nhân với hai mũ trừ b. Với b khoảng 0,29 thì nhân tư dữ liệu mua được vài điểm; với b gần không thì mọi khoản đầu tư dữ liệu trả về gần không.
●● Datasheet của tập dữ liệu	Tài liệu ghi nguồn, quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn, mức đồng thuận và các lớp bị thiếu. Danh sách trục không phủ được chính là danh sách chế độ hỏng đã biết trước.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chuẩn hoá đúng việc chương này yêu cầu: ghi lại quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn và các lớp bị thiếu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn
Dòng SAM và Grounding DINO trong quy trình gán nhãn voxel51/fiftyone	Kéo chi phí của mask và bbox xuống một bậc, tức dịch chuyển trực tiếp cột chi phí trong bảng chọn dạng đầu ra	Đã thành mặc định tính tới 9/2026
	Công cụ soi chênh lệch giữa khung lấy mẫu và quần thể bằng cách lát cắt dữ liệu theo thiết bị, ánh sáng, địa điểm	Hạ tầng, bền
ImageNet-v2 và WILDS	Bảng chứng đo được rằng độ chính xác trong khung lấy mẫu không dự đoán tốt độ chính xác ngoài nó	Kết quả đã công bố

MINI PROJECT SLAM • Biến “có nên làm keyframe không” thành một bài toán học máy • 1–2 ngày

Mục tiêu đi hết tám quyết định của chương trên một bài toán bạn đã có sẵn trực giác kỹ thuật. Mục đích là thấy rõ chỗ nào trực giác đó *không* đủ để thay một định nghĩa viết ra giấy.

Dữ liệu	hai chuỗi EuRoC, một dễ và một khó (ví dụ một chuỗi MH và một chuỗi V2), cùng nhật ký của ORB-SLAM3 ghi lại từng khung hình và quyết định keyframe hiện tại của heuristic có sẵn.
Các bước	viết một tài liệu định nghĩa bài toán gồm sáu phần — (1) đơn vị phân tích: một khung hình, một đoạn, hay một cặp khung hình so với keyframe gần nhất, và hệ quả của lựa chọn đó lên cách chia dữ liệu; (2) định nghĩa nhãn dương/âm kèm ít nhất năm ca biên viết rõ (khung hình mờ nhưng nhiều đặc trưng, camera đứng yên, quay nhanh tại chỗ, vừa mất bám xong, vùng đã lập bản đồ dày); (3) taxonomy: nhị phân hay ba mức, và mỗi mức dẫn tới hành động khác nhau nào trong hệ; (4) dạng đầu ra và metric đi kèm; (5) khung lấy mẫu so với quần thể thật của bạn, ghi rõ trục nào EuRoC không phủ; (6) giao thức chia dữ liệu chống rò rỉ theo chuỗi. Sau đó tự gán 300 khung hình và nhờ một người khác gán 80 khung hình trong số đó theo đúng tài liệu, không được hỏi lại.
Xong khi	hai người gán độc lập theo tài liệu của bạn mà không hỏi thêm một câu nào, và bạn báo cáo được mức đồng thuận trên 80 khung hình chung bằng một con số (κ hoặc tỉ lệ khớp). Ngưỡng để coi là đạt: sau một vòng sửa tài liệu, mức đồng thuận phải tăng, và bạn liệt kê được ít nhất ba ca biên mà vòng đầu không nghĩ ra.
Nối với	B/03 cho biết keyframe nằm ở khối nào trong luồng dữ liệu và độ khó bị đẩy đi đâu khi bạn thay heuristic bằng mô hình; B/06 biến quy trình gán nhãn này thành một quy trình có ngân sách; B/07 kiểm tra giao thức chia dữ liệu ở phần (6); B/04 dùng chính tập nhãn này để vẽ learning curve và quyết định có đáng gán tiếp không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nhãn không phải một sự thật có sẵn mà là kết quả của một định nghĩa, và mọi định nghĩa đều dễ hờ các ca biên: biên vật nhòe, mức che khuất, kích thước tối thiểu. Mô hình học từ tập nhãn đó nên không thể nhất quán hơn chính dữ liệu dạy nó một cách đáng tin.
- A2. Là chênh lệch giữa quần thể và khung lấy mẫu. Cả năm lần chia tập đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ năm lần thay vì đo thêm thứ mới; phần quần thể nằm ngoài khung không xuất hiện trong bất kỳ phép chia nào.
- A3. Vì nó đổi *đồng thời* kiến trúc, metric và toàn bộ nhãn đã gán. Kiến trúc thì huấn luyện lại là xong; nhãn thì phải làm lại từ đầu với chi phí lớn hơn nhiều lần, và đó là thứ thường giết dự án.
- A4. Có: huấn luyện cùng một cấu hình ở bốn tỉ lệ dữ liệu, khớp $e(n) \approx a n^{-b}$ và đọc số mũ b . Với $b \approx 0,29$, nhân tư dữ liệu mua được vài điểm; với b gần 0, mọi khoản đầu tư dữ liệu tiếp theo đều trả về gần không.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn nhờ người khác đi chợ hộ. Nếu mẫu giấy chỉ ghi “mua ít rau”, bạn sẽ nhận về thứ khác với thứ mình hình dung; nhờ người khác thì lại nhận về thứ thứ ba. Không ai sai cả — mẫu giấy sai. Bài toán gốc của chương này đúng như vậy, chỉ khác là người đi chợ là một cái máy. Trước khi có cái máy nào chạy, bạn phải viết mẫu giấy đó: mỗi lần nó nhìn vào cái gì, cái gì được coi là đúng, và những trường hợp lấp lửng thì tính sao. Rau hơi héo có mua không? Bó to gấp đôi thì tính là một hay hai?

Mẹo gồm ba việc. Một, viết các trường hợp lấp lửng ra giấy trước khi bắt đầu chứ không phải sau. Hai, nhờ hai người cùng đi chợ với cùng mẫu giấy rồi xem họ mua khác nhau bao nhiêu; chênh lệch đó chính là mức tốt nhất cái máy có thể đạt, vì nó chỉ học được từ những gì hai người kia mang về. Ba, nhớ rằng bạn chỉ nhờ mua ở chợ đầu ngõ nên sẽ không bao giờ biết chợ khác bán gì; nếu nhà sắp chuyển đi nơi khác thì việc phải làm là đi chợ mới, chứ không phải viết lại mẫu giấy cho hay hơn.

Cái giá phải trả là viết mẫu giấy vừa tốn thời gian vừa nhầm chán, và phải làm xong trước khi được chạm vào phần thú vị.

Chỗ không rút gọn được: câu “cái máy không bao giờ giỏi hơn hai người đi chợ” gần đúng nhưng không đúng hẳn. Nếu mỗi người nhầm theo một kiểu riêng, cái máy học từ cả hai có thể trung bình hoá được phần nhầm và nhỉnh hơn từng người. Nói gọn hơn nữa là sai.

Một câu: *Cái máy chỉ làm được đúng cái mẫu giấy bạn viết, nên phần khó nhất của dự án nằm ở tờ giấy chứ không nằm ở cái máy.*

1.8 Kết nối

- **Ngược lên** — ba câu hỏi đầu của B/03-giai-phau-he-thong (đầu vào, đầu ra, giám sát) ở đây được trả lời cho *bài toán của bạn* thay vì cho bài toán của người khác; và đôn bầy “thêm dữ liệu” trong bảng ba đôn bầy của B/04-nen-tang-hoc-tu-du-lieu ở đây có được quy trình đo cụ thể.
- **Đi tiếp** — B/06-chat-luong-du-lieu nhận vấn đề nhất quán nhãn ở bước hai và biến nó thành quy trình có ngân sách; B/07-chia-du-lieu-va-danh-gia nhận đơn vị phân tích ở bước một và cho thấy chọn sai nó sinh ra rò rỉ thế nào; B/08-dich-chuyen-phan-phoi xử lý phần quần thể nằm ngoài khung lấy mẫu.
- **Sang ngang** — việc chọn dạng đầu ra ràng buộc luôn thước đo, nên Phần E về đánh giá đọc dễ hơn nhiều nếu bạn quay lại Bảng 1.1 khi đọc nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chọn khung hình làm đơn vị phân tích rồi chia dữ liệu ngẫu nhiên lại tạo ra rò rỉ, và triệu chứng của rò rỉ đó hiện ra ở con số nào?
2. Vì sao mức bất đồng giữa hai người gán nhãn lại là trần hiệu năng thực tế, và trong hoàn cảnh nào mô hình vượt được trần đó?
3. Đưa ra một bài toán mà bbox không chỉ kém chính xác hơn mask mà còn sai về nguyên tắc. Sai ở đâu, và metric nào che giấu cái sai đó?
4. Một mô hình đạt kết quả ổn định qua năm lần chia tập rồi sập khi triển khai — cặp khái niệm quần thể / khung lấy mẫu giải thích hiện tượng đó thế nào, mà không cần dùng tới từ “overfitting”?
5. Vì sao reweighting không sửa được selection bias, và chính xác thì nó sửa được loại lệch nào, không sửa được loại nào?
6. Learning curve của bạn có số mũ $b \approx 0,05$. Kết luận gì, và bước tiếp theo là gì?
7. Trong bốn quyết định — đơn vị phân tích, taxonomy, dạng đầu ra, ngưỡng của metric — cái nào rẻ nhất để đổi sau khi đã gán nhãn xong, và vì sao?

Nguồn

- [1] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [2] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [3] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [4] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [5] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [6] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [7] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [8] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.